

Slip no 1

1) Write a PHP script to keep track of number of times the web page has been accessed (Use Session Tracking).

```
<!-- name- slip1.php -->
<?php
session_start();
if (!isset($_SESSION['access_count'])) {
    $_SESSION['access_count'] = 1;
} else {
    $_SESSION['access_count']++;
}
echo "This page has been accessed " . $_SESSION['access_count'] . " times.";
?>
```

2) Create 'Position_Salaries' Data set. Build a linear regression model by identifying independent and target variable. Split the variables into training and testing sets, then divide the training and testing sets into a 7:3 ratio, respectively and print them. Build a simple linear regression model.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

salary = pd.read_csv('Position_Salaries.csv')
print("Data sample:")
print(salary.sample(5))
X = salary[['Level']].values
y = salary['Salary'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
plt.figure(figsize=(10,6))
plt.scatter(X_test, y_test, color='green', label='Actual')
plt.plot(X_train, model.predict(X_train), color='red', linewidth=3, label='Predicted')
```

```
plt.title('Position Level vs Salary (Linear Regression)')
plt.xlabel('Position Level')
plt.ylabel('Salary')
plt.legend()
plt.show()
print("\nModel Evaluation:")
print(f"R-squared: {r2_score(y_test, y_pred):.2f}")
print(f"Mean Squared Error: {mean_squared_error(y_test, y_pred):.2f}")
print(f"Model Coefficients: {model.coef_[0]:.2f} (slope), {model.intercept_:.2f} (intercept)")
```

Slip no 2

1) Write a PHP script to change the preferences of your web page like font style, font size, font color, background color using cookie. Display selected setting on next web page and actual implementation (with new settings) on third page (Use Cookies).

```
<!-- Page 1 name- slip2.php -->
<?php
if (!isset($_POST['submit'])) {
    echo '<form method="post" action="">
        Font Style: <select name="font_style">
            <option value="Arial">Arial</option>
            <option value="Verdana">Verdana</option>
            <option value="Times New Roman">Times New Roman</option>
        </select><br>
        Font Size: <input type="number" name="font_size" min="10" max="30"><br>
        Font Color: <input type="color" name="font_color"><br>
        Background Color: <input type="color" name="bg_color"><br>
        <input type="submit" name="submit" value="Save Preferences">
    </form>';
} else {
    setcookie('font_style', $_POST['font_style'], time() + 86400);
    setcookie('font_size', $_POST['font_size'], time() + 86400);
    setcookie('font_color', $_POST['font_color'], time() + 86400);
    setcookie('bg_color', $_POST['bg_color'], time() + 86400);
    header("Location: display_preferences.php");
    exit();
}
```

```
}
?>
```

```
<!-- Page 2 name- display_preferences.php -->
<?php
echo "<h2>Your Selected Settings:</h2>";
echo "Font Style: ".$_COOKIE['font_style']."<br>";
echo "Font Size: ".$_COOKIE['font_size']."<br>";
echo "Font Color: ".$_COOKIE['font_color']."<br>";
echo "Background Color: ".$_COOKIE['bg_color']."<br>";
echo '<a href="apply_settings.php">Apply These Settings</a>';
?>
```

```
<!-- Page 3 name - apply_settings.php -->
<?php
// Page 3: Apply the settings
echo '<style>
    body {
        font-family: '.$_COOKIE['font_style'].';
        font-size: '.$_COOKIE['font_size'].'px;
        color: '.$_COOKIE['font_color'].';
        background-color: '.$_COOKIE['bg_color'].';
    }
</style>';
echo "<h2>Your Settings Have Been Applied!</h2>";
echo "<p>This page demonstrates your selected preferences.</p>";
?>
```

2) Create ‘Salary’ Data set . Build a linear regression model by identifying independent and target variable. Split the variables into training and testing sets and print them. Build a simple linear regression model for predicting purchases.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt
```

```

num_samples = 1000
salary_mean = 50000
salary_std = 10000
purchases_slope = 0.001
purchases_intercept = 10
salary = np.random.normal(salary_mean, salary_std, num_samples)
purchases = salary * purchases_slope + purchases_intercept + np.random.normal(0, 5,
num_samples)
data = pd.DataFrame({'Salary': salary, 'Purchases': purchases})
X = data[['Salary']]
y = data['Purchases']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
train_rmse = np.sqrt(mean_squared_error(y_train, model.predict(X_train)))
test_rmse = np.sqrt(mean_squared_error(y_test, model.predict(X_test)))

print("Training RMSE:", train_rmse)
print("Testing RMSE:", test_rmse)
plt.scatter(X_test, y_test, color='green')
plt.plot(X_train, model.predict(X_train), color='red', linewidth=3)
plt.title('Regression(Salary)')
plt.xlabel('Salary')
plt.ylabel('Purchases')
plt.show()

```

Slip no 3

1) Write a PHP script to accept username and password. If in the first three chances, username and password entered is correct then display second form with “Welcome message” otherwise display error message. [Use Session]

```
<!-- name- slip3.php -->
```

```
<?php
```

```
session_start();
```

```
// Correct credentials (in real app, store securely)
```

```
$valid_username = "admin";
```

```

$valid_password = "password123";

if (!isset($_SESSION['attempts'])) {
    $_SESSION['attempts'] = 0;
}

if (isset($_POST['submit'])) {
    if ($_POST['username'] == $valid_username && $_POST['password'] ==
$valid_password) {
        // Successful login
        unset($_SESSION['attempts']);
        header("Location: welcome.php");
        exit();
    } else {
        // Failed attempt
        $_SESSION['attempts']++;
        if ($_SESSION['attempts'] >= 3) {
            $error = "Maximum attempts reached. Please try again later.";
            session_destroy();
        } else {
            $remaining = 3 - $_SESSION['attempts'];
            $error = "Invalid credentials. You have $remaining attempts remaining.";
        }
    }
}

if (isset($_SESSION['attempts']) && $_SESSION['attempts'] >= 3) {
    echo "<h2>Error</h2>";
    echo "<p>You have exceeded the maximum number of login attempts.</p>";
} else {
    echo '<form method="post">
        <h2>Login</h2>';
    if (isset($error)) {
        echo "<p style='color:red>$error</p>";
    }
    echo '
        Username: <input type="text" name="username" required><br>
        Password: <input type="password" name="password" required><br>
        <input type="submit" name="submit" value="Login">
    </form>';
}
?>

<!-- welcome.php -->

```

```
<?php
session_start();
echo "<h2>Welcome!</h2>";
echo "<p>You have successfully logged in.</p>";
?>
```

2) Create 'User' Data set having 5 columns namely: User ID, Gender, Age, Estimated Salary and Purchased. Build a logistic regression model that can predict whether on the given parameter a person will buy a car or not.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

df = pd.read_csv('User_Data.csv')
df = pd.get_dummies(df, columns=['Gender'], drop_first=True)

X = df[['Gender_Male', 'Age', 'EstimatedSalary']]
y = df['Purchased']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=15)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

def predict_purchase(gender, age, salary, model):
    gender_num = 1 if gender.lower() == 'male' else 0
    input_data = pd.DataFrame([[gender_num, age, salary]], columns=['Gender_Male',
'Age', 'EstimatedSalary'])
    prediction = model.predict(input_data)
    return "Will Purchase" if prediction[0] == 1 else "Will Not Purchase"

gender = "Male"
age = 42
```

```
salary = 149000
result = predict_purchase(gender, age, salary, model)
```

Slip no 4

1) Write a PHP script to accept Employee details (Eno, Ename, Address) on first page. On second page accept earning (Basic, DA, HRA). On third page print Employee information (Eno, Ename, Address, Basic, DA, HRA, Total) [Use Session]

```
<!-- Page 1 name- slip4.php -->
```

```
<?php
session_start();
if (!isset($_POST['earnings'])) {
    echo '<form method="post">
        <h2>Employee Details</h2>
        Employee No: <input type="text" name="eno" required><br>
        Employee Name: <input type="text" name="ename" required><br>
        Address: <textarea name="address" required></textarea><br>
        <input type="submit" name="earnings" value="Next">
    </form>';
} else {
    // Store employee details in session
    $_SESSION['eno'] = $_POST['eno'];
    $_SESSION['ename'] = $_POST['ename'];
    $_SESSION['address'] = $_POST['address'];
    header("Location: earnings.php");
    exit();
}
?>
```

```
<!-- Page 2 name- earnings.php -->
```

```
<?php
session_start();
if (!isset($_POST['summary'])) {
    echo '<form method="post">
        <h2>Earnings Details</h2>
        Basic: <input type="number" name="basic" required><br>
        DA: <input type="number" name="da" required><br>
```

```

        HRA: <input type="number" name="hra" required><br>
        <input type="submit" name="summary" value="Next">
    </form>';
} else {
    $_SESSION['basic'] = $_POST['basic'];
    $_SESSION['da'] = $_POST['da'];
    $_SESSION['hra'] = $_POST['hra'];
    header("Location: summary.php");
    exit();
}
?>

```

```

<!-- Page 3 name- summary.php -->
<?php
session_start();
$total = $_SESSION['basic'] + $_SESSION['da'] + $_SESSION['hra'];
echo "<h2>Employee Information</h2>";
echo "<table border='1'>
    <tr><th>Employee No</th><td>".$_SESSION['eno']. "</td></tr>
    <tr><th>Employee Name</th><td>".$_SESSION['ename']. "</td></tr>
    <tr><th>Address</th><td>".$_SESSION['address']. "</td></tr>
    <tr><th>Basic</th><td>".$_SESSION['basic']. "</td></tr>
    <tr><th>DA</th><td>".$_SESSION['da']. "</td></tr>
    <tr><th>HRA</th><td>".$_SESSION['hra']. "</td></tr>
    <tr><th>Total</th><td>$total</td></tr>
</table>";
session_destroy();
?>

```

2) Build a simple linear regression model for Fish Species Weight Prediction.

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

```



```

fish_data = pd.read_csv('Fish.csv')
X = fish_data[['Width']]
y = fish_data['Weight']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("Mean Squared Error:", mse)
print("R-squared:", r2)
print("\nCoefficients:", model.coef_)
print("Intercept:", model.intercept_)
plt.scatter(X_test, y_test, color='green')
plt.plot(X_train, model.predict(X_train), color='red', linewidth=3)
plt.title('Weight vs Width Regression')
plt.xlabel('Width')
plt.ylabel('Weight')
plt.show()

```

Slip no 5

1) Create XML file named “Item.xml” with item-name, item-rate, item quantity Store the details of 5 Items of different Types

```

<?xml version="1.0" encoding="UTF-8"?>
<items>
  <item>
    <item-name>Laptop</item-name>
    <item-rate>50000</item-rate>
    <item-quantity>10</item-quantity>
  </item>
  <item>
    <item-name>Mobile</item-name>
    <item-rate>20000</item-rate>
    <item-quantity>20</item-quantity>
  </item>
  <item>

```

```

    <item-name>Tablet</item-name>
    <item-rate>15000</item-rate>
    <item-quantity>15</item-quantity>
  </item>
  <item>
    <item-name>Headphones</item-name>
    <item-rate>2000</item-rate>
    <item-quantity>30</item-quantity>
  </item>
  <item>
    <item-name>Keyboard</item-name>
    <item-rate>1000</item-rate>
    <item-quantity>25</item-quantity>
  </item>
</items>

```

2) Use the iris dataset. Write a Python program to view some basic statistical details like percentile, mean, std etc. of the species of 'Iris-setosa', 'Iris-versicolor' and 'Iris-virginica'. Apply logistic regression on the dataset to identify different species (setosa, versicolor, verginica) of Iris flowers given just 4 features: sepal and petal lengths and widths.. Find the accuracy of the model.

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

iris_data = pd.read_csv('Iris.csv')
X = iris_data[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']]
y = iris_data['Species']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy of the Logistic Regression model:", accuracy)

```

```
plt.figure(figsize=(10, 6))
plt.scatter(X_test['SepalLengthCm'], y_test, color='green', alpha=0.6, label='Actual')
plt.scatter(X_test['SepalLengthCm'], y_pred, color='red', alpha=0.6, marker='x',
label='Predicted')
plt.title('Logistic Regression (Iris Dataset)')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Species')
plt.legend()
plt.tight_layout()
plt.show()
```

Slip no 6

1) Write PHP script to read “book.xml” file into simpleXML object. Display attributes and elements . (simple_xml_load_file() function)

```
<!-- Page 1 name- slip6.php -->
<?php
$xml = simplexml_load_file("book.xml");
if ($xml === false) {
    echo "Failed to load XML file";
    exit;
}
echo "<h2>Book Details</h2>";
echo "<table
border='1'><tr><th>ID</th><th>Title</th><th>Author</th><th>Price</th></tr>";
foreach ($xml->book as $book) {
    echo "<tr>";
    echo "<td>".$book['id']. "</td>";
    echo "<td>".$book->title. "</td>";
    echo "<td>".$book->author. "</td>";
    echo "<td>".$book->price. "</td>";
    echo "</tr>";
}
echo "</table>";
?>
```

```
<!-- Page 2 name- book.xml -->
<books>
<book id="1">
<title>PHP Basics</title>
<author>John Doe</author>
<price>500</price>
</book>
</books>
```

2) Create the following dataset in python & Convert the categorical values into numeric format. Apply the apriori algorithm on the above dataset to generate the frequent itemsets and association rules. Repeat the process with different min_sup values.

```
import numpy as np
import pandas as pd
from mlxtend.frequent_patterns import apriori,
association_rules
from mlxtend.preprocessing import TransactionEncoder

transactions = [
    ['Bread', 'Milk'],
    ['Bread', 'Diaper', 'Beer', 'Eggs'],
    ['Milk', 'Diaper', 'Beer', 'Coke'],
    ['Bread', 'Milk', 'Diaper', 'Beer'],
    ['Bread', 'Milk', 'Diaper', 'Coke']
]
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)
df = df.astype(int) # Convert boolean to int (1/0)
freq_items = apriori(df, min_support=0.5,
use_colnames=True)
print("Frequent Itemsets:\n", freq_items)
rules = association_rules(freq_items, metric='support',
min_threshold=0.05)
rules = rules.sort_values(['support', 'confidence'],
ascending=[False, False])
print("\nAssociation Rules:\n", rules)
```

Slip no 7

1) Write a PHP script to read “Movie.xml” file and print all MovieTitle and ActorName of file using DOMDocument Parser. “Movie.xml” file should contain following information with at least 5 records with values. MovieInfoMovieNo, MovieTitle, ActorName ,ReleaseYear

```
<!-- Page 1 name- slip7.xml -->
<?php
$dom = new DOMDocument();
$dom->load('Movie.xml');
$movies = $dom->getElementsByTagName('movie');
echo "<h2>Movie Details</h2>";
echo "<table border='1'><tr><th>Movie Title</th><th>Actor Name</th></tr>";
foreach ($movies as $movie) {
    $title = $movie->getElementsByTagName('MovieTitle')->item(0)->nodeValue;
    $actor = $movie->getElementsByTagName('ActorName')->item(0)->nodeValue;
    echo "<tr><td>$title</td><td>$actor</td></tr>";
}
echo "</table>";
?>
```

```
<!-- Page 2 name- Movie.xml -->
<movies>
  <movie>
    <MovieNo>1</MovieNo>
    <MovieTitle>Inception</MovieTitle>
    <ActorName>Leonardo DiCaprio</ActorName>
    <ReleaseYear>2010</ReleaseYear>
  </movie>
  ... (at least 5 records)
</movies>
```

2) Download the Market basket dataset. Write a python program to read the dataset and display its #information. Preprocess the data (drop null values etc.) Convert the categorical values into numeric format. Apply the apriori algorithm on the above dataset to generate the frequent itemsets and association rules

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
```

```

df = pd.read_csv('Market_Basket_Optimisation.csv', header=None)
sample_size = min(50, len(df))
df = df.sample(sample_size, random_state=42)
transactions = []
for i in range(len(df)):
    transactions.append([str(item) for item in df.iloc[i] if str(item) != 'nan'])
from mlxtend.preprocessing import TransactionEncoder
te = TransactionEncoder()
te_array = te.fit_transform(transactions)
df1 = pd.DataFrame(te_array, columns=te.columns_)
freq_items = apriori(df1, min_support=0.005, use_colnames=True)
print("Frequent Itemsets:\n", freq_items.head())
rules = association_rules(freq_items, metric='support', min_threshold=0.005)
rules = rules.sort_values(['support', 'confidence'], ascending=[False, False])
print("\nTop Association Rules:\n", rules.head())

```

Slip no 8

1) Write a JavaScript to display message 'Exams are near, have you started preparing for?' (use alert box) and Accept any two numbers from user and display addition of two number. (Use Prompt and confirm box)

```

<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Examples</title>
  <script>
    // Display alert on page load
    window.onload = function() {
      alert("Exams are near, have you started preparing for?");

      // Number addition
      let num1 = parseFloat(prompt("Enter first number:"));
      let num2 = parseFloat(prompt("Enter second number:"));

      if (confirm("Do you want to see the sum?")) {
        alert(`The sum is: ${num1 + num2}`);
      } else {

```

```

        alert("Calculation cancelled");
    }
};
</script>
</head>
<body>
    <h1>JavaScript Examples</h1>
    <p>Check the alert, prompt and confirm boxes.</p>
</body>
</html>

```

2) Download the groceries dataset. Write a python program to read the dataset and display its information. Preprocess the data (drop null values etc.) Convert the categorical values into numeric format. Apply the apriori algorithm on the above dataset to generate the frequent itemsets and association rules.

```

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder

df = pd.read_csv('groceries.csv')
df = df.sample(min(50,len(df)), random_state=42)
transactions = []
for i in range(len(df)):
    transactions.append([str(df.values[i,j]) for j in range(len(df.columns)) if
str(df.values[i,j]) != 'nan'])
te = TransactionEncoder()
te_array = te.fit_transform(transactions)
te_array = te_array.astype('int')
df1 = pd.DataFrame(te_array, columns=te.columns_)
freq_items = apriori(df1, min_support=0.2, use_colnames=True)
print("Frequent Itemsets:")
print(freq_items)
rules = association_rules(freq_items, metric='support', min_threshold=0.2)
rules = rules.sort_values(['support','confidence'], ascending=[False,False])
print("\nTop Association Rules:")
print(rules.head())

```

1) Write a JavaScript function to validate username and password for a membership form

```
<!DOCTYPE html>
<html>
<head>
  <title>Membership Form Validation</title>
  <script>
    function validateForm() {
      const username = document.getElementById("username").value;
      const password = document.getElementById("password").value;

      if (username.length < 4) {
        alert("Username must be at least 4 characters long");
        return false;
      }

      if (password.length < 6) {
        alert("Password must be at least 6 characters long");
        return false;
      }

      alert("Form submitted successfully!");
      return true;
    }
  </script>
</head>
<body>
  <h1>Membership Form</h1>
  <form onsubmit="return validateForm()">
    Username: <input type="text" id="username" required><br>
    Password: <input type="password" id="password" required><br>
    <input type="submit" value="Register">
  </form>
</body>
</html>
```


2) Create your own transactions dataset and apply the above process on your dataset.

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
transaction=[['sugar','tea'], ['coffee','tea','sugar'], ['tea','coffee'], ['coffee','suagr','tea','milk']]

from mlxtend.preprocessing import TransactionEncoder
te=TransactionEncoder()
te_array=te.fit(transaction).transform(transaction)
df=pd.DataFrame(te_array,columns=te.columns_)
df = df.astype(int)
df

freq_items=apriori(df,min_support=0.5,use_colnames=True)
rules=association_rules(freq_items,metric='support',min_threshold=0.05)
rules=rules.sort_values(['support','confidence'],ascending=[False,False])
rules
```

Slip no 10

1) Create a HTML file to insert text before and after a Paragraph using jQuery. [Hint : Use before() and after()]

```
<!DOCTYPE html>
<html>
<head>
  <title>jQuery Before and After</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <style>
    .added { color: green; font-style: italic; }
  </style>
  <script>
    $(document).ready(function() {
      // Insert text before and after paragraph
      $("p").before("<p class='added'>This text is inserted BEFORE the
paragraph</p>");
      $("p").after("<p class='added'>This text is inserted AFTER the paragraph</p>");
    });
```

```

</script>
</head>
<body>
  <h1>jQuery Before and After Example</h1>
  <p>This is the original paragraph.</p>
</body>
</html>

```

2) Create the following dataset in python & Convert the categorical values into numeric format. Apply the apriori algorithm on the above dataset to generate the frequent itemsets and association rules. Repeat the process with different min_sup values.

```

import numpy as np
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

transactions = [['eggs','milk','bread'], ['eggs','apple'], ['milk','bread'], ['apple',
'milk'], ['milk','apple','bread']]

from mlxtend.preprocessing import TransactionEncoder
te=TransactionEncoder()
te_array=te.fit(transactions).transform(transactions)
df=pd.DataFrame(te_array, columns=te.columns_)
df = df.astype(int)
df

freq_items = apriori(df, min_support = 0.5, use_colnames = True)
print(freq_items)

rules = association_rules(freq_items, metric ='support', min_threshold=0.05)
rules = rules.sort_values(['support', 'confidence'], ascending =[False,False])
rules

```

Slip no 11

1) Write a Javascript program to accept name of student, change font color to red, font size to 18 if student name is present otherwise on clicking on empty text box display image which changes its size (Use onblur, onload, onmouseover, onmouseleave, onmouseup)

```
<!DOCTYPE html>
```

```

<html>
<head>
  <title>Student Name Form</title>
  <style>
    .highlight { color: red; font-size: 18px; }
    #studentImage { width: 100px; transition: width 0.5s; }
  </style>
  <script>
    function checkName() {
      const name = document.getElementById("studentName").value;
      const nameField = document.getElementById("studentName");

      if (name.trim() !== "") {
        nameField.className = "highlight";
      }
    }
    function resizeImage() {
      const img = document.getElementById("studentImage");
      img.style.width = (img.width === 200) ? "100px" : "200px";
    }
  </script>
</head>
<body>
  <h1>Student Information</h1>
  <form>
    Student Name: <input type="text" id="studentName" onblur="checkName()"><br>
    
  </form>
</body>
</html>

```

2) Create the following dataset in python & Convert the categorical values into numeric format. Apply the apriori algorithm on the above dataset to generate the frequent itemsets and association rules. Repeat the process with different min_sup values.

```

import numpy as np
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

transactions = [['eggs','milk','bread'], ['eggs','apple'], ['milk','bread'], ['apple',
'milk'], ['milk','apple','bread']]

from mlxtend.preprocessing import TransactionEncoder
te=TransactionEncoder()
te_array=te.fit(transactions).transform(transactions)
df=pd.DataFrame(te_array, columns=te.columns_)
df = df.astype(int)
df

freq_items = apriori(df, min_support = 0.5, use_colnames = True)
print(freq_items)

rules = association_rules(freq_items, metric='support', min_threshold=0.05)
rules = rules.sort_values(['support', 'confidence'], ascending=[False,False])
rules= rules.sort_values(['support', 'confidence'], ascending=[False,False])

```

Slip no 12

1) Write AJAX program to read contact.dat file and print the contents of the file in a tabular format when the user clicks on print button. Contact.dat file should contain srno, name, residence number, mobile number, Address.

```

<!-- Page 1 name- Slip12.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Contact Information</title>
  <script>
    function loadContacts() {
      const xhr = new XMLHttpRequest();
      xhr.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
          displayContacts(this.responseText);
        }
      }
    }
  </script>

```

```

    };
    xhr.open("GET", "contact.dat", true);
    xhr.send();
}
function displayContacts(data) {
    const contacts = data.split('\n');
    let tableHTML = "<table border='1'><tr><th>SNo</th><th>Name</th><th>Residence  
No</th><th>Mobile No</th><th>Address</th></tr>";
    for (let contact of contacts) {
        if (contact.trim() !== "") {
            const fields = contact.split(',');
            tableHTML += `<tr>
                <td>${fields[0]}</td>
                <td>${fields[1]}</td>
                <td>${fields[2]}</td>
                <td>${fields[3]}</td>
                <td>${fields[4]}</td>
            </tr>`;
        }
    }
    tableHTML += "</table>";
    document.getElementById("contactTable").innerHTML = tableHTML;
}
</script>
</head>
<body>
    <h1>Contact Information</h1>
    <button onclick="loadContacts()">Print Contacts</button>
    <div id="contactTable"></div>
</body>
</html>

```

```
<!-- Page 2 name- contact.dat - ->
```

```

1,John Doe,022-1234567,9876543210,123 Main St
2,Jane Smith,022-7654321,9876123450,456 Oak Ave
3,Bob Johnson,022-1122334,8765432109,789 Pine Rd

```

2) Create ‘heights-and-weights’ Data set . Build a linear regression model by identifying independent and target variable. Split the variables into training and testing sets and print them. Build a simple linear regression model for predicting purchases.

```

import numpy as np
import pandas as pd

```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt
num_samples = 1000
heights = np.random.normal(170, 10, num_samples)
weights = 0.5 * heights + 30 + np.random.normal(0, 5, num_samples)
data = pd.DataFrame({'Height': heights, 'Weight': weights})
X = data[['Height']]
y = data['Weight']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
train_rmse = np.sqrt(mean_squared_error(y_train, model.predict(X_train)))
test_rmse = np.sqrt(mean_squared_error(y_test, model.predict(X_test)))
print("Training RMSE:", train_rmse)
print("Testing RMSE:", test_rmse)
plt.scatter(X_test, y_test, color='green')
plt.plot(X_train, model.predict(X_train), color='red', linewidth=3)
plt.title('Regression(Height vs Weight)')
plt.xlabel('Height')
plt.ylabel('Weight')
plt.show()

```

Slip no 13

1) Write AJAX program where the user is requested to write his or her name in a text box, and the server keeps sending back responses while the user is typing. If the user name is not entered then the message displayed will be, “Stranger, please tell me your name!”. If the name is Rohit, Virat, Dhoni, Ashwin or Harbhajan , the server responds with “Hello, master !”. If the name is anything else, the message will be “, I don’t know you!”

```
<!-- Page 1 name- slip13.html - ->
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Name Validation</title>
```

```
<script>
```

```

function checkName() {
    const name = document.getElementById("nameInput").value;
    const xhr = new XMLHttpRequest();

    xhr.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
            document.getElementById("response").innerHTML = this.responseText;
        }
    };

    xhr.open("GET", "name_check.php?name=" + encodeURIComponent(name),
true);
    xhr.send();
}
</script>
</head>
<body>
    <h1>Name Checker</h1>
    Enter your name: <input type="text" id="nameInput" onkeyup="checkName()">
    <div id="response"></div>
</body>
</html>

```

```

<!-- Page 2 name- name_check.php - ->
<?php
$name = $_GET['name'] ?? "";
$masters = ['Rohit', 'Virat', 'Dhoni', 'Ashwin', 'Harbhajan'];
if (empty($name)) {
    echo "Stranger, please tell me your name!";
} elseif (in_array(ucfirst($name), $masters)) {
    echo "Hello, master!";
} else {
    echo "$name, I don't know you!";
}
?>

```

2) download nursery dataset from UCI. Build a linear regression model by identifying independent #d target variable. Split the variables into training and testing sets and print them. Build a

simple linear regression model for predicting purchases.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

nursery_data = pd.read_csv(r'C:\Users\faimp\OneDrive\Documents\CSV\nursery.csv')
X = nursery_data[['social']]
y = nursery_data[['health']]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
train_rmse = np.sqrt(mean_squared_error(y_train, model.predict(X_train)))
test_rmse = np.sqrt(mean_squared_error(y_test, model.predict(X_test)))
print("Training RMSE:", train_rmse)
print("Testing RMSE:", test_rmse)
plt.scatter(X_test, y_test, color='green')
plt.plot(X_train, model.predict(X_train), color='red', linewidth=3)
plt.title('Regression (Social vs Health)')
plt.xlabel('Social')
plt.ylabel('Health')
plt.show()
```

Slip no 14

1) Create TEACHER table as follows TEACHER(tno, tname, qualification, salary). Write Ajax program to select a teachers name and print the selected teachers details

```
<!-- Page 2 name- slip14.html - ->
<!DOCTYPE html>
<html>
<head>
  <title>Teacher Details</title>
  <script>
    function getTeacherDetails() {
```



```

const teacherName = document.getElementById("teacherSelect").value;
const xhr = new XMLHttpRequest();

xhr.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        document.getElementById("teacherDetails").innerHTML =
this.responseText;
    }
};

xhr.open("GET", "get_teacher.php?name=" +
encodeURIComponent(teacherName), true);
xhr.send();
}
</script>
</head>
<body>
<h1>Teacher Information System</h1>
Select Teacher:
<select id="teacherSelect" onchange="getTeacherDetails()">
    <option value="">--Select--</option>
    <option value="John Smith">John Smith</option>
    <option value="Mary Johnson">Mary Johnson</option>
    <option value="David Lee">David Lee</option>
</select>
<div id="teacherDetails"></div>
</body>
</html>

<!-- Page 1 name- get_teacher.php - ->
<?php
    $teachers = [
        'John Smith' => ['tno' => '101', 'qualification' => 'PhD', 'salary' => '80000'],
        'Mary Johnson' => ['tno' => '102', 'qualification' => 'MSc', 'salary' => '75000'],
        'David Lee' => ['tno' => '103', 'qualification' => 'MBA', 'salary' => '85000']
    ];
    $name = $_GET['name'] ?? "";
    if (!empty($name) && isset($teachers[$name])) {
        $t = $teachers[$name];
        echo "<h3>Teacher Details</h3>";
        echo "<table border='1'>";
        echo "<tr><th>Teacher No</th><td>{$t['tno']}</td></tr>";
    }
}

```

```

echo "<tr><th>Name</th><td>{$name}</td></tr>";
echo "<tr><th>Qualification</th><td>{$t['qualification']}</td></tr>";
echo "<tr><th>Salary</th><td>{$t['salary']}</td></tr>";
echo "</table>";
} else {
    echo "Please select a teacher";
}
?>

```

2) Create the following dataset in python & Convert the categorical values into numeric format. Apply the apriori algorithm on the above dataset to generate the frequent itemsets and association rules. Repeat the process with different min_sup values.

```

import numpy as np
import pandas as pd
from mlxtend.frequent_patterns import apriori,
association_rules
transactions =
[['eggs', 'milk', 'bread'], ['eggs', 'apple'], ['milk', 'bread'], ['apple',
'milk'], ['milk', 'apple', 'bread']]

```

```

from mlxtend.preprocessing import TransactionEncoder
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)
df = df.astype(int)

```

```

freq_items = apriori(df, min_support=0.5,
use_colnames=True)
print(freq_items)

```

```

rules = association_rules(freq_items, metric='support',
min_threshold=0.05)
rules = rules.sort_values(['support', 'confidence'],
ascending=[False, False])
print(rules)

```

Slip no 15

1) Write Ajax program to fetch suggestions when user is typing in a textbox. (eg like google suggestions. Hint create array of suggestions and matching string will be displayed)

```

<!DOCTYPE html>
<html>
<head>
  <title>Autocomplete Example</title>
  <script>
    const suggestions = [
      "Apple", "Banana", "Cherry", "Date", "Elderberry",
      "Fig", "Grape", "Honeydew", "Kiwi", "Lemon"
    ];
    function showSuggestions() {
      const input = document.getElementById("searchBox").value.toLowerCase();
      const suggestionList = document.getElementById("suggestions");
      suggestionList.innerHTML = "";
      if (input.length === 0) return;
      const matches = suggestions.filter(fruit =>
        fruit.toLowerCase().startsWith(input)
      );
      matches.forEach(fruit => {
        const li = document.createElement("li");
        li.textContent = fruit;
        li.onclick = function() {
          document.getElementById("searchBox").value = fruit;
          suggestionList.innerHTML = "";
        };
        suggestionList.appendChild(li);
      });
    }
  </script>
  <style>
    #suggestions {
      list-style: none;
      padding: 0;
      margin: 0;
      border: 1px solid #ccc;
      width: 200px;
    }
    #suggestions li {

```

```

padding: 5px;
cursor: pointer;
}
#suggestions li:hover {
background-color: #f0f0f0;
}
</style>
</head>
<body>
<h1>Fruit Search</h1>
<input type="text" id="searchBox" onkeyup="showSuggestions()" placeholder="Start
typing...">
<ul id="suggestions"></ul>
</body>
</html>

```

2) Create the following dataset in python & Convert the categorical values into numeric format. Apply the apriori algorithm on the above dataset to generate the frequent itemsets and association rules. Repeat the process with different min_sup values

```

import numpy as np
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
transactions = [['eggs','milk','bread'], ['eggs','apple'], ['milk','bread'], ['apple', 'milk'],
['milk','apple','bread']]
from mlxtend.preprocessing import TransactionEncoder
te=TransactionEncoder()
te_array=te.fit(transactions).transform(transactions)
df=pd.DataFrame(te_array, columns=te.columns_)
df = df.astype(int)
df
freq_items = apriori(df, min_support = 0.5, use_colnames = True)
print(freq_items)
rules = association_rules(freq_items, metric ='support', min_threshold=0.05)
rules = rules.sort_values(['support', 'confidence'], ascending =[False,False])

```

Slip no 17

1) Write a Java Script Program to show Hello Good Morning message onload event using alert box

and display the Student registration form.

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration</title>
  <script>
    window.onload = function() {
      alert("Hello Good Morning!");
    };

    function validateForm() {
      const name = document.getElementById("name").value;
      const email = document.getElementById("email").value;

      if (name.trim() === "" || email.trim() === "") {
        alert("Please fill in all fields");
        return false;
      }

      alert("Registration successful!");
      return true;
    }
  </script>
</head>
<body>
  <h1>Student Registration Form</h1>
  <form onsubmit="return validateForm()">
    Name: <input type="text" id="name" required><br>
    Email: <input type="email" id="email" required><br>
    Course:
    <select id="course">
      <option value="cs">Computer Science</option>
      <option value="it">Information Technology</option>
      <option value="ece">Electronics</option>
    </select><br>
```

```

        <input type="submit" value="Register">
    </form>
</body>
</html>

```

2) Consider text paragraph. So, keep working. Keep striving. Never give up. Fall down seven times, get up eight. Ease is a greater threat to progress than hardship. Ease is a greater threat to progress than hardship. So, keep moving, keep growing, keep learning. See you at work. Preprocess the text to remove any special characters and digits. Generate the summary using extractive summarization process.

```

import re
from nltk.tokenize import sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

```

```

text = "So, keep working. Keep striving. Never give up. Fall down seven times, get up
eight. Ease is a greater threat to progress than hardship. Ease is a greater threat to
progress than hardship. So, keep moving, keep growing, keep learning. See you at work."

```

```

def preprocess_text(text):
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    text = re.sub(r'\d+', '', text)
    return text.lower()

def tokenize_sentences(text):
    return sent_tokenize(text)

preprocessed_text = preprocess_text(text)
sentences = tokenize_sentences(preprocessed_text)
stop_words = set(stopwords.words("english"))
stemmer = PorterStemmer()

def preprocess_sentence(sentence):
    words = sentence.split()
    words = [stemmer.stem(word) for word in words if word not in stop_words]
    return ' '.join(words)

preprocessed_sentences = [preprocess_sentence(sentence) for sentence in sentences]
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(preprocessed_sentences)

```

```

cosine_sim_matrix = cosine_similarity(tfidf_matrix, tfidf_matrix)
sentence_scores = cosine_sim_matrix.sum(axis=1)
sorted_indices = sentence_scores.argsort()[::-1]
num_sentences_summary = 2
summary_sentences = [sentences[idx] for idx in
sorted_indices[:num_sentences_summary]]
summary = ''.join(summary_sentences)
print("Original Text:\n", text)
print("\nExtractive Summary:\n", summary)

```

Slip no 18

1) Write a Java Script Program to print Fibonacci numbers on onclick event.

```

<!DOCTYPE html>
<html>
<head>
  <title>Fibonacci Sequence</title>
  <script>
    function generateFibonacci() {
      const n = parseInt(prompt("How many Fibonacci numbers to generate?", 10));

      if (isNaN(n) || n <= 0) {
        alert("Please enter a positive number");
        return;
      }

      let fib = [0, 1];
      for (let i = 2; i < n; i++) {
        fib[i] = fib[i-1] + fib[i-2];
      }

      // Display first n numbers
      fib = fib.slice(0, n);

      document.getElementById("result").innerHTML =

```

```

`<h3>First ${n} Fibonacci Numbers:</h3><p>${fib.join(', ')}</p>`;
    }
</script>
</head>
<body>
    <h1>Fibonacci Sequence Generator</h1>
    <button onclick="generateFibonacci()">Generate Fibonacci Numbers</button>
    <div id="result"></div>
</body>
</html>

```

2) Consider any text paragraph. Remove the stopwords. Tokenize the paragraph to extract words and sentences. Calculate the word frequency distribution and plot the frequencies. Plot the wordcloud of the text.

```

import re
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from collections import Counter

text = """
Hello world this is 4 and Here to summarize text
"""

stop_words = set(stopwords.words('english'))
filtered_text = ' '.join([word for word in re.findall(r'\b\w+\b', text.lower()) if
word not in stop_words])
words = word_tokenize(filtered_text)
sentences = sent_tokenize(text)
word_freq = Counter(words)

plt.figure(figsize=(10, 6))
plt.bar(word_freq.keys(), word_freq.values())
plt.xlabel('Words')
plt.ylabel('Frequency')
plt.title('Word Frequency Distribution')
plt.xticks(rotation=45)
plt.show()

```



```
wordcloud = WordCloud(width=800, height=400,
background_color='white').generate(filtered_text) plt.figure(figsize=(10, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Wordcloud')
plt.show()
```

Slip no 19

1)Write a Java Script Program to validate user name and password on onSubmit event

```
<!DOCTYPE html>
<html>
<head>
  <title>Login Form Validation</title>
  <script>
    function validateForm() {
      const username = document.getElementById("username").value;
      const password = document.getElementById("password").value;

      if (username === "" || password === "") {
        alert("Both username and password are required!");
        return false;
      }

      if (username.length < 4) {
        alert("Username must be at least 4 characters long");
        return false;
      }

      if (password.length < 6) {
        alert("Password must be at least 6 characters long");
        return false;
      }

      alert("Login successful!");
```

```

        return true;
    }
</script>
</head>
<body>
    <h1>Login Form</h1>
    <form onsubmit="return validateForm()">
        Username: <input type="text" id="username" required><br>
        Password: <input type="password" id="password" required><br>
        <input type="submit" value="Login">
    </form>
</body>
</html>

```

**2) Download the movie_review.csv dataset from Kaggle by using the following link
: https://www.kaggle.com/nltkdata/movie-review/version/3?select=movie_review.csv to perform
sentiment analysis on above dataset and create a wordcloud**

```

import pandas as pd
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from wordcloud import WordCloud
import matplotlib.pyplot as plt

nltk.download('wordnet')

df = pd.read_csv('CSV/movie_review.csv')
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess_text(text):
    words = word_tokenize(text)
    words = [word.lower() for word in words if word.isalpha()]
    words = [lemmatizer.lemmatize(word) for word in words if word not in stop_words]
    return ' '.join(words)

```

```
df['clean_text'] = df['text'].apply(preprocess_text)
all_text = ' '.join(df['clean_text'])
```

```
wordcloud = WordCloud(width=800, height=400,
background_color='white').generate(all_text)
```

```
plt.figure(figsize=(10, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud of Movie Reviews')
plt.show()
```

Slip no 20

1)create a student.xml file containing at least 5 student information

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
  <student>
    <id>101</id>
    <name>John Smith</name>
    <age>21</age>
    <course>Computer Science</course>
    <grade>A</grade>
  </student>
  <student>
    <id>102</id>
    <name>Emily Johnson</name>
    <age>20</age>
    <course>Information Technology</course>
    <grade>B+</grade>
  </student>
  <student>
    <id>103</id>
    <name>Michael Brown</name>
    <age>22</age>
    <course>Electronics</course>
```

```

    <grade>A-</grade>
</student>
<student>
    <id>104</id>
    <name>Sarah Davis</name>
    <age>21</age>
    <course>Mechanical Engineering</course>
    <grade>B</grade>
</student>
<student>
    <id>105</id>
    <name>David Wilson</name>
    <age>20</age>
    <course>Computer Science</course>
    <grade>A</grade>
</student>
</students>

```

2) Consider text paragraph."""Hello all, Welcome to Python Programming Academy. Python #Programming Academy is a nice platform to learn new programming skills. It is difficult to get enrolled in this Academy."""**Remove the stopwords.**

```

import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
text = """Hello all, Welcome to Python Programming Academy. Python Programming
Academy is a nice platform to learn new programming skills. It is difficult to get enrolled
in this Academy."""
nltk.download('stopwords')
nltk.download('punkt')
words = word_tokenize(text)
stop_words = set(stopwords.words('english'))
filtered_words = [word for word in words if word.lower() not in stop_words]
filtered_text = ' '.join(filtered_words)

print("Original Text:\n", text)
print("\nText after removing stopwords:\n", filtered_text)

```

Slip no 21

1)Add a JavaScript File in Codeigniter. The Javascript code should check whether a number is positive or negative.

Name - number_check.php

```
<!DOCTYPE html>
<html>
<head>
  <title>Number Check</title>
  <script src="<?php echo base_url('assets/js/number_check.js'); ?>"></script>
</head>
<body>
  <h1>Number Sign Checker</h1>
  <form onsubmit="return checkNumber()">
    Enter a number: <input type="number" id="numberInput" required>
    <input type="submit" value="Check">
  </form>
  <div id="result"></div>
</body>
</html>
```

Name - number_check.js

```
function checkNumber() {
  const number = parseFloat(document.getElementById("numberInput").value);
  const resultDiv = document.getElementById("result");

  if (number > 0) {
    resultDiv.innerHTML = `<p>${number} is a positive number</p>`;
  } else if (number < 0) {
    resultDiv.innerHTML = `<p>${number} is a negative number</p>`;
  } else {
    resultDiv.innerHTML = `<p>The number is zero</p>`;
  }
}
```

```

    return false; // Prevent form submission
}

```

application/controllers/NumberCheck.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class NumberCheck extends CI_Controller {
    public function index() {
        $this->load->view('number_check');
    }
}

```

2) Build a simple linear regression model for User Data.

```

import re
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from collections import Counter
import nltk

nltk.download('punkt')
nltk.download('stopwords')
text = """Hello world this is 4 and Here to summarize text"""
stop_words = set(stopwords.words('english'))
filtered_text = ' '.join([word for word in re.findall(r'\b\w+\b', text.lower()) if word not in
stop_words])
words = word_tokenize(filtered_text)
sentences = sent_tokenize(text)
word_freq = Counter(words)
plt.figure(figsize=(10, 6))
plt.bar(word_freq.keys(), word_freq.values())
plt.xlabel('Words')
plt.ylabel('Frequency')
plt.title('Word Frequency Distribution')

```

```

plt.xticks(rotation=45)
plt.show()
wordcloud = WordCloud(width=800, height=400,
background_color='white').generate(filtered_text)
plt.figure(figsize=(10, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud')
plt.show()

```

Slip no 22

1) Create a table student having attributes(rollno, name, class). Using codeigniter, connect to the database and insert 5 records in it.

Name - Student.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
class Student extends CI_Controller {
```

```
    public function __construct() {
```

```
        parent::__construct();
```

```
        $this->load->database();
```

```
        $this->load->helper('url');
```

```
    }
```

```
    public function index() {
```

```
        // Create table if not exists
```

```
        $this->db->query("CREATE TABLE IF NOT EXISTS student (
```

```
            rollno INT PRIMARY KEY,
```

```
            name VARCHAR(100),
```

```
            class VARCHAR(50)
```

```
        );
```

```
        // Insert sample records
```

```
        $data = [
```

```
            ['rollno' => 1, 'name' => 'John Smith', 'class' => 'CS'],
```

```
            ['rollno' => 2, 'name' => 'Emily Johnson', 'class' => 'IT'],
```

```
            ['rollno' => 3, 'name' => 'Michael Brown', 'class' => 'CS'],
```

```
            ['rollno' => 4, 'name' => 'Sarah Davis', 'class' => 'ME'],
```

```
            ['rollno' => 5, 'name' => 'David Wilson', 'class' => 'IT']
```

```
        ];
```

```

$this->db->insert_batch('student', $data);

    echo "Student records inserted successfully!";
}
}

```

2) Consider any text paragraph. Remove the stopwords.

```

import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

text = """Hello all, Welcome to Python Programming Academy. Hello all, Welcome to
Python Programming Academy. Hello all, Welcome to Python Programming
Academy. Hello all, Welcome to Python Programming Academy. Hello all, Welcome
to Python Programming Academy. Hello all, Welcome to Python Programming
Academy. Python Programming Academy is a nice platform to learn new
programming skills. It is difficult to get enrolled in this Academy."""

nltk.download('stopwords')
nltk.download('punkt')

words = word_tokenize(text)
stop_words = set(stopwords.words('english'))
filtered_words = [word for word in words if word.lower() not in stop_words]
filtered_text = ' '.join(filtered_words)

print("Original Text:\n", text)
print("\nText after removing stopwords:\n", filtered_text)

```

Slip no 23

1) Create a table student having attributes(rollno, name, class) containing atleast 5 recodes. Using codeigniter, display all its records.

Name - Student.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Student extends CI_Controller {
    public function __construct() {

```



```

    parent::__construct();
    $this->load->database();
    $this->load->helper('url');
}
public function index() {
    $query = $this->db->get('student');
    $data['students'] = $query->result();
    $this->load->view('student_list', $data);
}
}

```

Name - student_list.php

```

<!DOCTYPE html>
<html>
<head>
    <title>Student List</title>
    <style>
        table { border-collapse: collapse; width: 50%; }
        th, td { border: 1px solid #ddd; padding: 8px; text-align: left; }
        th { background-color: #f2f2f2; }
    </style>
</head>
<body>
    <h1>Student Records</h1>
    <table>
        <tr>
            <th>Roll No</th>
            <th>Name</th>
            <th>Class</th>
        </tr>
        <?php foreach ($students as $student): ?>
            <tr>
                <td><?php echo $student->rollno; ?></td>
                <td><?php echo $student->name; ?></td>
                <td><?php echo $student->class; ?></td>
            </tr>

```

```

        <?php endforeach; ?>
    </table>
</body>
</html>

```

2) Consider any text paragraph. Preprocess the text to remove any special characters and digit.

```

import re
text = "hello 1234 this is @"
def preprocess_text(text):
    text = re.sub(r'[^\a-zA-Z\s]', "", text)
    text = re.sub(r'\d+', "", text)
    return text.lower()
preprocessed_text = preprocess_text(text)
print("Original Text:\n", text)
print("\nAfter processing text:\n", preprocessed_text)

```

Slip no 24

1) Write a PHP script to create student.xml file which contains student roll no, name, address, college and course. Print students detail of specific course in tabular format after accepting course as input.

```

<?php
if (!file_exists('student.xml')) {
    $students = [
        ['rollno' => '101', 'name' => 'John Smith', 'address' => '123 Main St', 'college' =>
        'ABC College', 'course' => 'CS'],
        ['rollno' => '102', 'name' => 'Emily Johnson', 'address' => '456 Oak Ave', 'college' =>
        'XYZ College', 'course' => 'IT'],
        ['rollno' => '103', 'name' => 'Michael Brown', 'address' => '789 Pine Rd', 'college' =>
        'ABC College', 'course' => 'ME'],
        ['rollno' => '104', 'name' => 'Sarah Davis', 'address' => '321 Elm St', 'college' =>
        'DEF College', 'course' => 'CS'],
        ['rollno' => '105', 'name' => 'David Wilson', 'address' => '654 Maple Ave', 'college'
        => 'XYZ College', 'course' => 'IT']
    ];

    $xml = new SimpleXMLElement('<students></students>');

    foreach ($students as $student) {

```

```

$studentNode = $xml->addChild('student');
$studentNode->addChild('rollno', $student['rollno']);
$studentNode->addChild('name', $student['name']);
$studentNode->addChild('address', $student['address']);
$studentNode->addChild('college', $student['college']);
$studentNode->addChild('course', $student['course']);
}
$xml->asXML('student.xml');
}

if (isset($_GET['course'])) {
    $course = $_GET['course'];
    $xml = simplexml_load_file('student.xml');

    echo "<h2>Students in $course Course</h2>";
    echo "<table border='1'><tr><th>Roll  
No</th><th>Name</th><th>Address</th><th>College</th></tr>";

    foreach ($xml->student as $student) {
        if ((string)$student->course == $course) {
            echo "<tr>
                <td>{$student->rollno}</td>
                <td>{$student->name}</td>
                <td>{$student->address}</td>
                <td>{$student->college}</td>
            </tr>";
        }
    }

    echo "</table>";
} else {
    echo '<form method="get">
        Enter course to filter: <input type="text" name="course" required>
        <input type="submit" value="Filter">
    </form>';
}
?>

```

2) Consider the following dataset : <https://www.kaggle.com/datasets/datasnaek/youtube-new?select=INvideos.csv> Write a Python script for the following : i. Read the dataset and perform data cleaning operations on it. ii. Find the total views, total likes, total dislikes and comment count.

```
import pandas as pd
import matplotlib.pyplot as plt
file_path = 'CSV/INvideos.csv' # Modify this if your file is in a different folder
try:
    data = pd.read_csv(file_path)
except FileNotFoundError:
    print(f"Error: The file '{file_path}' was not found. Please check the path and try again.")
    exit()

data.dropna(inplace=True)

total_views = data['views'].sum()
total_likes = data['likes'].sum()
total_dislikes = data['dislikes'].sum()
total_comments = data['comment_count'].sum()

print("Total Views:", total_views)
print("Total Likes:", total_likes)
print("Total Dislikes:", total_dislikes)
print("Total Comments:", total_comments)
least_liked_video = data.loc[data['likes'].idxmin()]
top_liked_video = data.loc[data['likes'].idxmax()]

least_commented_video = data.loc[data['comment_count'].idxmin()]
top_commented_video = data.loc[data['comment_count'].idxmax()]

print("\nLeast Liked Video:", least_liked_video)
print("Top Liked Video:", top_liked_video)
print("Least Commented Video:", least_commented_video)
print("Top Commented Video:", top_commented_video)

plt.scatter(data['views'], data['likes'], color='blue', alpha=0.5)
plt.title('Views vs Likes')
```

```
plt.xlabel('Views')
plt.ylabel('Likes')
plt.show()
```

Slip no 25

1) Write a script to create “cricket.xml” file with multiple elements as shown below: Write a script to add multiple elements in “cricket.xml” file of category, country=“India”.

1.

```
<?php
// Create cricket.xml if it doesn't exist
if (!file_exists('cricket.xml')) {
    $xml = new SimpleXMLElement('<CricketTeam></CricketTeam>');

    // Add Australia team
    $team = $xml->addChild('Team');
    $team->addAttribute('country', 'Australia');
    $team->addChild('player', 'David Warner');
    $team->addChild('runs', '5000');
    $team->addChild('wicket', '15');

    $xml->asXML('cricket.xml');
}

// Add India team
$xml = simplexml_load_file('cricket.xml');
$team = $xml->addChild('Team');
$team->addAttribute('country', 'India');
$team->addChild('player', 'Virat Kohli');
$team->addChild('runs', '8000');
$team->addChild('wicket', '10');

$xml->asXML('cricket.xml');

echo "cricket.xml has been updated with India team details.";
?>
```

2) Consider the following dataset : https://www.kaggle.com/datasets/seungguini/youtube-comments-for-covid19-relatedvideos?select=covid_2021_1.csv Write a Python script for the following : i. Read the dataset and perform data cleaning operations on it. ii. Tokenize the comments in words. iii. Perform sentiment analysis and find the percentage of positive, negative and neutral comments.

```
import pandas as pd
import re
from textblob import TextBlob

data = pd.read_csv('CSV/covid_2021_1.csv')

# Drop rows with missing 'comment_text'
data = data.dropna(subset=['comment_text'])

# Clean the comments
data['clean_comment'] = data['comment_text'].apply(lambda x: re.sub(r'^a-zA-Z\s', '', str(x)))
data['clean_comment'] = data['clean_comment'].apply(lambda x: re.sub(r'\s+', ' ', str(x)))
# Fixed to replace multiple spaces with one
data['tokenized_comment'] = data['clean_comment'].apply(lambda x: x.split())

# Initialize counters for sentiment
positive_comments = 0
negative_comments = 0
neutral_comments = 0

# Analyze sentiment
for comment in data['clean_comment']:
    analysis = TextBlob(comment)
    if analysis.sentiment.polarity > 0:
        positive_comments += 1
    elif analysis.sentiment.polarity < 0:
        negative_comments += 1
    else:
        neutral_comments += 1

# Calculate the total number of comments
total_comments = len(data)
```

```
# Calculate percentages
```

```
ps_per = (positive_comments / total_comments) * 100
```

```
neg_per = (negative_comments / total_comments) * 100
```

```
neut_per = (neutral_comments / total_comments) * 100
```

```
# Print results
```

```
print("Percentage of positive comments: ", format(ps_per, '.2f'))
```

```
print("Percentage of negative comments: ", format(neg_per, '.2f'))
```

```
print("Percentage of neutral comments: ", format(neut_per, '.2f'))
```

Slip no 26

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Employee Details</title>
```

```
<script>
```

```
function getEmployeeDetails() {
```

```
    const empName = document.getElementById("empSelect").value;
```

```
    const xhr = new XMLHttpRequest();
```

```
    xhr.onreadystatechange = function() {
```

```
        if (this.readyState == 4 && this.status == 200) {
```

```
            document.getElementById("empDetails").innerHTML = this.responseText;
```

```
        }
```

```
    };
```

```
    xhr.open("GET", "get_employee.php?name=" + encodeURIComponent(empName), true);
```

```
    xhr.send();
```

```
}
```

```
</script>
```

```
</head>
```

```
<body>
```

```
<h1>Employee Information System</h1>
```

```
Select Employee:
```

```
<select id="empSelect" onchange="getEmployeeDetails()">
```

```
<option value="">-- Select --</option>
```

```
<option value="John Smith">John Smith</option>
```

```
<option value="Emily Johnson">Emily Johnson</option>
```

```

        <option value="Michael Brown">Michael Brown</option>
    </select>

    <div id="empDetails"></div>
</body>
</html>

<!-- get_employee.php -->
<?php
// Database connection
$conn = new mysqli("localhost", "username", "password", "company_db");

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

$empName = $_GET['name'] ?? '';

if (!empty($empName)) {
    $stmt = $conn->prepare("SELECT * FROM EMP WHERE ename = ?");
    $stmt->bind_param("s", $empName);
    $stmt->execute();
    $result = $stmt->get_result();

    if ($result->num_rows > 0) {
        $row = $result->fetch_assoc();
        echo "<table border='1'>
            <tr><th>Employee No</th><td>".$row['eno']. "</td></tr>
            <tr><th>Name</th><td>".$row['ename']. "</td></tr>
            <tr><th>Designation</th><td>".$row['designation']. "</td></tr>
            <tr><th>Salary</th><td>".$row['salary']. "</td></tr>
        </table>";
    } else {
        echo "No employee found with that name.";
    }
    $stmt->close();
}

$conn->close();
?>

```